RACK-LEVEL MICRO-CLIMATE AND GROWTH OPTIMIZER FOR OYSTER MUSHROOM FARMS

INDIVIDUAL THESIS

Amana Ismath

(IT22156624)

Individual component: Image-Based Oyster Mushroom Disease Detection and Rack-Level Contamination Tracking

Group ID: R26-ISE-002

Report submitted in partial fulfilment of the requirements for the

Bachelor of Science (Honours) Degree in Information Technology

Specialization in Information Systems Engineering

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Amana Ismath

Student ID: IT22156624

Signature:  Date: 10/10/2026

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: Ms. Narmada Gamage		
Co-supervisor: Ms. Rangi Liyanage		

ABSTRACT

Oyster mushroom cultivation is an important agricultural activity for small- and medium-scale farmers in Sri Lanka. However, fungal contamination can reduce crop quality and productivity, while manual visual inspection may delay the identification of infected cultivation bags. Existing disease detection approaches have limitations in distinguishing specific contamination types and associating detected infections with individual rack locations. This research focuses on developing a mobile-assisted, image-based disease detection and rack-level contamination tracking system for oyster mushroom cultivation. The main objective is to develop and evaluate a YOLOv8-based object detection model capable of identifying visible signs of green mold (Trichoderma) and black mold (Aspergillus niger). The research adopts a Design Science Research approach supported by experimental evaluation, including image dataset preparation, annotation, model training, and system integration. The proposed solution uses React Native for the mobile interface, FastAPI for backend communication, and SQLite for storing rack information and contamination records. The disease detection component achieved a reported detection performance of over 90% in identifying green mold and black mold, demonstrating the potential of deep learning for automated mushroom disease monitoring. The system also supports the visual localization of suspected contamination and the association of detection records with relevant rack and cultivation-bag identifiers. The individual contribution includes dataset preparation, disease detection model development, visual localization, application integration, rack-level contamination tracking, and technical evaluation. The findings indicate that image-based disease detection can support more consistent identification of visible fungal contamination and provide farmers with useful information for targeted monitoring. Further evaluation under diverse real-world cultivation conditions is necessary to establish the system's generalizability and long-term practical effectiveness.

Keywords: Oyster Mushroom Disease Detection, YOLOv8, Rack-Level Contamination Tracking, Smart Agriculture

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to my supervisor, Ms. Narmada Gamage, and my co-supervisor, Ms. Rangi Liyanage, for their guidance, and valuable feedback throughout my individual research and development work.

I would also like to extend my appreciation to the Agricultural Research Center in Makandura for their valuable assistance with collecting images of oyster mushroom cultivation bags and supporting the testing of the proposed disease detection system. Their cooperation and practical insights into oyster mushroom cultivation were important in developing and evaluating the image-based disease detection component of this research.

Furthermore, I am grateful to the academic staff of the Sri Lanka Institute of Information Technology (SLIIT) for the knowledge and academic support provided throughout my undergraduate studies. I would also like to acknowledge the researchers whose published work contributed to my understanding of computer vision, deep learning, and disease detection.

Finally, I would like to thank my family and friends for their encouragement, patience, and support throughout my research journey.

TABLE OF CONTENTS

1 INTRODUCTION 1
1.1 Overall project context 1
1.2 Individual component context 2
1.3 Problem statement 3
1.4 Research gap 4
1.5 Aim and objectives 6
1.6 Research questions or hypotheses 7
1.7 Scope limitations and assumptions 8
1.7.1 Scope 8
1.7.2 Limitations 9
1.7.3 Assumptions 10
1.8 Individual contribution 11
1.9 Thesis structure 12
2 LITERATURE REVIEW AND RELATED WORK 13
2.1 Review method and scope 13
2.2 Domain and theoretical foundations 14
2.2.1 Oyster Mushroom Cultivation and Fungal Contamination 14
2.2.2 Computer Vision in Agricultural Disease Detection 15
2.2.3 Image Classification and Object Detection 16
2.2.4 YOLO-Based Object Detection 17
2.2.5 Rack-Level Contamination Tracking 18
2.3 Related research 19
2.4 Existing systems products and techniques 20
2.5 Gap synthesis and conceptual framework 21

3 RESEARCH METHODOLOGY 24
3.1 Research design 24
3.2 Data participants or study materials 24
3.3 Experimental or study procedure 26
3.4 Measures and evaluation metrics 28
3.5 Data analysis 29
3.6 Ethics privacy security and responsible AI 30
3.7 Reproducibility and research artefacts 31
4 PRODUCT AND SOLUTION DESIGN 32
4.1 Requirements 32
4.1.1 Functional Requirements 32
4.1.2 Non-functional Requirements 32
4.2 Component architecture within the complete solution 33
4.3 Design alternatives and decisions 35
4.4 Detailed design 36
4.4.1 Model Inference 36
4.4.2 API Communication 36
4.4.3 Database Operations 36
4.4.4 Configuration and Integration 37
4.4.5 Interaction Sequence 37
4.5 Standards and professional practice 38
5 IMPLEMENTATION INTEGRATION AND TESTING 40
5.1 Development environment and technology stack 40
5.2 Component implementation 40
5.2.1 Model Preparation and Training 40
5.2.2 Backend Implementation 41

5.2.3 Database Implementation 41
5.2.4 Mobile Application Interaction 42
5.3 Integration with the other components 43
5.4 Verification and software testing 43
5.5 Deployment and operation 44
6 RESULTS AND VALIDATION 45
6.1 Research results 45
6.2 Product and technical results 45
6.3 User or stakeholder validation 46
6.4 Integration results 46
6.5 Summary of findings 46
7 DISCUSSION 48
7.1 Interpretation of findings 48
7.2 Comparison with literature baselines and existing solutions 48
7.3 Research contribution 49
7.4 Product contribution 49
7.5 Limitations and threats to validity 50
7.6 Implications 50
8 CONCLUSION AND FUTURE DIRECTIONS 52
8.1 Conclusion 52
8.2 Achievement of objectives 52
8.3 Individual research and product contributions 53
8.4 Future directions 54
REFERENCES 56
GLOSSARY 58
APPENDICES 61

Appendix A Supervisor's Acceptance Form - Data 61

Appendix B Preliminary Model Training and Evaluation Results 62

Appendix C Functional Test Cases 62

Appendix D Academic integrity and AI use disclosure 63

LIST OF FIGURES

Figure 1 Conceptual Framework of the Mushroom Disease Detection and Rack-Level Tracking Component..... 22
Figure 2 Component Architecture of the YOLOv8-Based Mushroom Disease Detection System 34
Figure 3 Sequence diagram of image submission, detection and rack-level update.. 38
Figure 4 Preliminary UI mockups: (a) Capture Image, (b) History and Reports, (c) Rack Tracking 42
Figure 5 Supervisor's Acceptance Form – Data, Research Project 2026 61
Figure 6 Preliminary Confusion Matrix 62

LIST OF TABLES

Table 1 Summary of Datasets Used 25
Table 2 Information stored by the component 37
Table 3 Technologies used in the component 40
Table 4 Reported Performance of the Disease Detection Model 45
Table 5 Product and Technical Performance Results 46
Table 6 Summary of Functional Test Cases 63

LIST OF ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
AWS	Amazon Web Services
CNN	Convolutional Neural Network
CO_2	Carbon Dioxide
FN	False Negative
FP	False Positive
FR	Functional Requirement
IoU	Intersection over Union
LSTM	Long Short-Term Memory
mAP	Mean Average Precision
ML	Machine Learning
REST	Representational State Transfer
RQ	Research Question
SQL	Structured Query Language
TC	Test Case
TP	True Positive
UAT	User Acceptance Testing
YOLOv8	You Only Look Once, Version 5
YOLOv8	You Only Look Once, Version 8

1 INTRODUCTION

1.1 Overall project context

Oyster mushroom cultivation has become a useful agricultural activity for small- and medium-scale farmers because it can be carried out within a relatively limited space and does not require the same level of land investment as many conventional crops. In Sri Lanka, this form of cultivation provides opportunities for income generation while contributing to the availability of nutritious food. However, maintaining a healthy cultivation environment requires careful attention to mushroom growth, temperature, humidity, carbon dioxide concentration, and the presence of diseases.

Traditional mushroom farming practices often depend on farmers' observations and periodic manual inspections. Although practical, these methods can make it difficult to identify problems consistently across cultivation areas containing multiple racks and large numbers of mushroom bags. Environmental conditions may also vary between rack positions, meaning that observations made in one area may not accurately represent the conditions elsewhere in the growing facility. When problems are not identified promptly, farmers may face avoidable production losses and additional management work.

The research project titled Rack-Level Micro-Climate and Growth Optimizer for Oyster Mushroom Farms addresses these challenges through an integrated system intended to support monitoring and decision-making in oyster mushroom cultivation. The overall project combines four individual research components: biological growth-stage detection and harvest estimation, moisture-stress regulation using temperature and humidity data, carbon dioxide-related physiological stress prediction, and image-based disease detection with rack-level contamination tracking.

The purpose of combining these components is to provide a more comprehensive view of cultivation conditions than would be available through isolated observations or

manual records alone. Each component focuses on a particular aspect of mushroom production while contributing to the wider objective of improving farm monitoring and supporting more informed management decisions.

Within this overall project, the present thesis focuses specifically on image-based oyster mushroom disease detection and rack-level contamination tracking. It investigates the use of computer vision and deep learning to identify visible signs of selected fungal contamination in cultivation bags and connect the resulting detections with relevant rack records.

1.2 Individual component context

Disease identification is an important part of oyster mushroom farm management because contaminated cultivation bags may affect surrounding bags and create additional work for farmers. Identifying potentially infected bags at an early stage can help farmers investigate affected areas and decide which management actions may be necessary. However, the usefulness of an inspection process depends on how consistently visible symptoms can be recognized and how effectively the findings are recorded.

The individual component examined in this thesis focuses on two target disease categories: green mold, associated in the project with Trichoderma, and black mold, associated with Aspergillus niger. The research investigates whether an image-based object detection model can recognize visible signs associated with these categories in photographs of oyster mushroom cultivation bags. The intended system also aims to indicate the approximate locations of regions detected within an image, allowing users to examine the model's output rather than receiving only a general classification result.

The proposed approach uses the YOLOv8 object detection framework to analyze images and identify relevant visual patterns. Images collected from oyster mushroom cultivation settings are prepared and annotated to provide examples for model

development and evaluation. The Agricultural Research Center in Makandura has supported the research through image collection and testing activities, helping connect the technical work with practical cultivation conditions.

The component is designed to operate through a mobile-assisted workflow. A user captures or selects an image using a mobile application, and the image is processed through a backend service that communicates with the detection model. React Native is proposed for the mobile interface, FastAPI for backend communication, and SQLite for storing rack information and contamination records. These technologies support the intended workflow from image submission to the presentation and recording of detection results.

An important aspect of this component is the relationship between disease detection and rack-level monitoring. Detecting a suspected infection in an image is only one part of the intended process. Associating that result with a particular cultivation bag and rack can help users maintain records of affected areas and review their condition over time.

1.3 Problem statement

Oyster mushroom farmers need to identify contaminated cultivation bags and keep track of affected areas to manage disease-related risks within their growing facilities. When inspections rely mainly on manual observation, identifying visible symptoms across numerous bags can require considerable time and consistent attention. Differences in lighting, viewing angle, symptom appearance, and image quality may also make visual assessment more difficult. If inspection findings are not recorded systematically, it can be challenging to determine which bags have been affected and where further inspection may be required.

Image-based detection offers a potential way to support this process by analyzing photographs and highlighting visual patterns associated with selected contamination

categories. However, a useful solution must address more than the ability to produce a prediction. It must also distinguish the target categories required by the application, indicate the locations of suspected contamination where possible, and present the results in a form that can be associated with cultivation records.

For the present research, the problem is therefore defined as the need for a practical, image-based approach to detecting visible signs of green mold and black mold in oyster mushroom cultivation bags and connecting the resulting detections with rack-level contamination records. The research must examine whether a YOLOv8-based model can perform this task with acceptable detection performance under the image conditions represented in the collected dataset, while supporting its use through a mobile application and backend service.

The problem is not limited to model development. A detection result that cannot be reviewed or linked to a relevant cultivation location may have limited practical value to a farmer. Consequently, the individual component also considers how detected regions, user confirmation, and rack-health records can be incorporated into a coherent workflow.

This thesis investigates the development and evaluation of such a workflow. Its effectiveness is assessed through model-performance measures and system testing, while recognizing that successful detection in a research dataset does not, by itself, establish reliable performance across all mushroom farms or cultivation conditions.

1.4 Research gap

Research into smart agriculture has explored the use of sensors, automated monitoring, and data-driven techniques to improve agricultural management. In mushroom cultivation, these approaches can support the observation of environmental conditions and help farmers identify changes that may affect production. Image analysis and deep

learning also offer opportunities to examine visible characteristics of cultivation bags and identify patterns that may be associated with disease.

Nevertheless, environmental monitoring and disease identification address different requirements. A system that measures temperature, humidity, or carbon dioxide does not necessarily identify visible fungal contamination. Similarly, a model that distinguishes broadly between healthy and unhealthy samples may not provide the specific disease categories required for a targeted inspection workflow. These distinctions are important when the intended application requires farmers to review suspected green mold and black mold separately.

The project proposal identifies the need for a solution that focuses on selected fungal contamination categories relevant to the intended Sri Lankan oyster mushroom farming context. It also identifies rack-level contamination tracking as an important requirement. In this setting, the practical value of a detection system depends not only on the model's predictions but also on whether the results can be associated with cultivation locations and maintained as usable records.

The research gap addressed by this thesis is therefore the need to develop and evaluate an integrated, image-based approach that combines detection of the selected contamination categories, visual localization of suspected infected regions, and rack-level record keeping within a mobile-assisted workflow. The component is intended to connect these functions rather than treating image classification as an isolated task.

The contribution of this research will be assessed in relation to the dataset collected, the performance of the trained model, the handling of image variation, and the successful integration of detection results with the application workflow. This evaluation is important because the suitability of the proposed approach cannot be established solely from the choice of a deep-learning framework. Its practical value

must be considered in light of the data used, the evaluation results, and the requirements of the intended users.

1.5 Aim and objectives

The aim of this research is to design, develop, and evaluate a mobile-assisted, YOLOv8-based image detection system that identifies visible signs of green mold and black mold in oyster mushroom cultivation bags, localizes suspected contamination within images, and supports rack-level contamination tracking.

The research is guided by the following objectives:

- Prepare a relevant image dataset. Collect and organize images of healthy oyster mushroom cultivation bags and bags showing visible signs associated with green mold and black mold, with support from the Agricultural Research Center in Makandura. Annotate the images appropriately for supervised object detection.

- Develop the disease detection model. Train and optimize a YOLOv8-based object detection model to recognize the selected contamination categories and distinguish them from healthy cultivation bags, according to the defined dataset labels.

- Support visual localization of suspected contamination. Configure the detection workflow to identify and display the approximate image regions associated with model predictions, enabling users to inspect the results.

- Integrate the model into a mobile-assisted workflow. Develop the necessary communication between the mobile application, backend service, and

detection model so that images can be submitted for analysis and the resulting predictions can be returned to the user.

- Implement rack-level contamination tracking. Support the association of detection records with relevant cultivation-bag and rack identifiers, including a mechanism for users to confirm or record rack-health information.

- Evaluate model and system performance. Assess the detection model using appropriate performance measures, including precision and recall, and examine relevant system behavior such as response time and the reliability of the image-to-record workflow.

The research proposal establishes a target detection performance of at least 95% precision and recall. This target is treated as an evaluation objective rather than an assumed outcome. The final assessment must report on the actual measured results and explain any differences between the target and the performance achieved.

1.6 Research questions or hypotheses

This research is guided by four research questions.

RQ1: How effectively can a YOLOv8-based object detection model identify visible signs of green mold and black mold in oyster mushroom cultivation-bag images?

This question examines the model's ability to detect the target categories using the prepared and annotated dataset. Performance will be assessed using appropriate detection metrics, including precision and recall, together with other relevant measures where available.

RQ2: How does variation in image-capture conditions affect the detection of suspected contamination?

Images may differ in lighting, viewing angle, distance, and the visibility of symptoms. This question examines the extent to which such variation influences detection performance within the collected dataset and the conditions represented during testing.

RQ3: How can image-based disease detection be integrated with a mobile application to support visual inspection and rack-level contamination tracking?

This question focuses on the technical workflow connecting image submission, model inference, presentation of detection results, and the storage of contamination records. It also considers how users can review and record the health status of relevant cultivation areas.

RQ4: To what extent does the implemented component meet its intended performance and functional requirements?

This question evaluates the developed component against the defined objectives and requirements. It considers model performance, response time, the presentation of detection results, and the correct association of records with cultivation-bag and rack identifiers.

1.7 Scope limitations and assumptions

1.7.1 Scope

This research covers the design, implementation, and evaluation of an image-based disease detection component for oyster mushroom cultivation. Its main technical focus is the use of YOLOv8 to identify visible signs associated with green mold and black mold in photographs of cultivation bags.

The scope includes the collection and organization of relevant images, annotation of the dataset, model training and optimization, and evaluation using a held-out test dataset. It also includes the presentation of detection results and visual localization of

suspected contamination. Where supported by the implementation, these results are connected to cultivation-bag and rack identifiers to maintain contamination records.

The proposed software workflow uses React Native for the mobile interface, FastAPI for backend communication, and SQLite for storing rack and contamination information. Testing includes the interaction between these elements and the handling of image submissions and detection results.

The research is limited to the individual disease detection and contamination-tracking component of the wider Rack-Level Micro-Climate and Growth Optimizer for Oyster Mushroom Farms project. The other group components, including biological growth-stage detection, moisture-stress regulation, and carbon dioxide-related stress prediction, provide the broader project context but are not the principal subject of this thesis.

1.7.2 Limitations

The model's performance depends on the quality, quantity, diversity, and accuracy of the collected and annotated images. If the dataset does not adequately represent different cultivation conditions, the model may perform less reliably on images that differ from those used during training and testing.

The target categories are restricted to the contamination classes defined for this research. The system is not intended to identify every disease or contaminant that may occur in mushroom cultivation. In addition, image-based predictions indicate visual patterns learned from the dataset and should not be treated as definitive laboratory confirmation of the organism responsible for a suspected infection.

The evaluation is also limited by the conditions under which the model and integrated application are tested. Performance measured on the available test dataset may not directly represent performance in every farm, growing environment, or device

configuration. Further testing with additional farms and varied real-world conditions may be necessary before broader deployment can be recommended.

Rack-level tracking depends on the availability and accuracy of the identifiers and records entered into the system. Incorrect or incomplete records may reduce the usefulness of contamination monitoring even when the image detector produces a reasonable prediction. The research therefore considers detection performance and record-handling functionality as related but distinct aspects of the system.

1.7.3 Assumptions

The research makes the following assumptions:

- Images collected for the study represent relevant examples of healthy cultivation bags and bags showing visible signs associated with the selected contamination categories.

- Dataset annotations are prepared consistently enough to support supervised model training and meaningful evaluation.

- The visual characteristics present in the collected images provide useful information for distinguishing the defined target categories.

- Users can provide or select the relevant cultivation-bag and rack identifiers when contamination records are created or updated.

- The mobile application, backend service, model, and database can communicate under the conditions available during testing.

- Users will treat model predictions as decision-support information and will review or confirm suspected contamination where the workflow requires it.

These assumptions define the conditions under which the proposed component is developed and evaluated. Their influence should be considered when interpreting the results, particularly when discussing whether the system could be applied beyond the dataset and testing environment used in this study.

1.8 Individual contribution

This thesis documents the individual contribution assigned to Amana Ismath (IT22156624) within the group research project Rack-Level Micro-Climate and Growth Optimizer for Oyster Mushroom Farms (Project ID: R26-ISE-002). The allocated component concerns image-based detection and classification of selected oyster mushroom diseases, visual localization of suspected contamination, and rack-level contamination tracking.

The individual research work covers the preparation of an image dataset containing healthy samples and samples showing visible signs associated with green mold and black mold. This includes organizing the collected images and preparing annotations for model development. Image collection and testing activities were supported by the Agricultural Research Center in Makandura.

A further contribution is the development and evaluation of the YOLOv8-based disease detection model. This work involves training and refining the model, examining its ability to identify the defined target categories, and assessing its performance using appropriate evaluation measures. The work also considers the display of detected regions so that users can examine the locations associated with model predictions.

The implementation contribution includes connecting the detection model to the proposed mobile-assisted workflow through the backend service. The component is designed to receive image submissions, return detection results, and support the storage of relevant rack and contamination information. The rack-level tracking functionality is intended to help users associate suspected infections with the appropriate cultivation locations and maintain records for subsequent review.

The individual evaluation covers the model's measured detection performance and the behavior of the integrated component during testing. This includes examining the handling of image submissions, the presentation of predictions, and the association of contamination records with the correct identifiers. The results are used to identify the extent to which the component meets its objectives and to document remaining limitations.

1.9 Thesis structure

This thesis is organized into eight chapters, each addressing a different stage of research, from defining the problem to evaluating the completed component and identifying opportunities for further development.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

This chapter examines research and technologies relevant to image-based disease detection in oyster mushroom cultivation. The review focuses on how computer vision, deep learning, and digital monitoring systems have been applied to identify disease symptoms and support farm management. Attention is given to the detection of fungal contamination in cultivation bags and the practical requirements of recording affected locations within rack-based growing environments.

The literature was examined from three related perspectives. The first concerns oyster mushroom cultivation and the difficulties associated with recognizing visible fungal contamination. The second addresses the technical methods used for image classification and object detection, including convolutional neural networks and the YOLO family of models. The third considers how detection methods are incorporated into agricultural monitoring systems and whether their outputs can support decisions at the level of individual cultivation bags or racks.

The review draws on studies discussed in the project proposal, including work on IoT-based oyster mushroom monitoring in Sri Lanka, YOLOv5-based disease detection, low-cost monitoring systems, and integrated smart farming concepts. These studies were considered in relation to their research objectives, detection approaches, system functions, and reported limitations. The intention is not simply to identify which study used the most recent technology, but to understand which approaches address the requirements of the present research and which questions remain unresolved.

For the individual component, the most relevant considerations are the ability to distinguish selected contamination categories, localize suspected disease regions within images, operate with photographs collected under practical cultivation conditions, and connect detection results with rack-level records. Studies concerned

mainly with environmental sensing or automatic climate control are included where they help establish the wider smart farming context, but they are not treated as direct alternatives to image-based disease detection.

2.2 Domain and theoretical foundations

2.2.1 Oyster Mushroom Cultivation and Fungal Contamination

Oyster mushrooms, commonly associated with the genus Pleurotus, are cultivated using organic substrates that support the development of fungal mycelium and fruiting bodies. Successful cultivation depends on maintaining suitable growing conditions and monitoring the condition of the cultivation bags throughout the production cycle.

In rack-based cultivation, multiple bags are placed close to one another to make efficient use of available space. This arrangement can simplify farm organization, but it also increases the importance of regular inspection. When a bag develops visible signs of contamination, farmers need to recognize the problem and identify its location before deciding how to manage the affected area.

Fungal contamination is particularly relevant because unwanted organisms may develop on the substrate or other exposed surfaces of a cultivation bag. The present research focuses on two selected categories: green mold, associated with Trichoderma, and black mold, associated with Aspergillus niger. These categories were selected in the approved project proposal as the target conditions for the image-based detection component.

The visible appearance of contamination is important for image-based analysis. Changes in color, texture, and the distribution of affected regions may provide useful visual features for a detection model. However, these features can also be influenced by lighting, image resolution, viewing angle, and the stage at which the image is

captured. A dark area in a photograph, for example, may result from shadow rather than fungal contamination.

This distinction creates a challenge for automated detection. A model must learn useful visual differences from the training data while avoiding excessive reliance on unrelated characteristics of the images. Furthermore, recognizing a visual pattern does not necessarily establish the biological identity of the organism present. For this reason, the proposed system is considered a tool for identifying suspected contamination rather than a replacement for laboratory diagnosis.

2.2.2 Computer Vision in Agricultural Disease Detection

Computer vision enables software systems to extract and analyze information from digital images. In agriculture, image-based techniques have been investigated for tasks such as identifying crop conditions, recognizing visible disease symptoms, monitoring growth, and supporting automated inspection.

Traditional image-processing methods commonly depend on manually selected visual characteristics, including color distributions, shapes, edges, and textures. Such techniques can be useful when the appearance of the target is relatively consistent. However, their performance may become less reliable when images contain complex backgrounds, inconsistent illumination, or substantial variation in the appearance of the object being examined.

Deep learning provides a different approach by allowing models to learn image features from labelled examples. Rather than depending entirely on predefined visual rules, a trained neural network identifies patterns that are useful for the task represented in its training data.

Convolutional neural networks (CNNs) have been widely used in image analysis because they can learn spatial features through successive processing layers. In a

disease-recognition application, a CNN may learn combinations of colour, texture, and shape that help distinguish images belonging to different categories.

The usefulness of these models nevertheless depends on the available training data. Images that contain inconsistent labels, insufficient variation, or repeated examples from closely related capture sessions may produce misleading evaluation results. A model can perform well on familiar images without demonstrating the same level of reliability on images collected in a different cultivation environment.

2.2.3 Image Classification and Object Detection

Image classification and object detection are related to computer-vision tasks, but they produce different types of output.

An image classification model assigns a category to an entire image. In a simple mushroom disease application, it might predict that a photograph represents a healthy or diseased sample. This can provide a useful initial indication of health status, particularly when the image contains one clearly visible subject.

Object detection extends this process by identifying both a category and its approximate position within an image. A detection model can return bounding boxes that indicate where relevant visual features have been identified. This is useful when a photograph contains multiple objects or when the location of a suspected infected region is important to the user.

For oyster mushroom cultivation, this distinction has practical implications. A farmer may need to know not only whether an image contains signs of contamination, but also which part of the image has caused the model to report a possible infection. Bounding-box predictions can help users inspect the suspected region and decide whether the corresponding cultivation bag requires further attention.

However, bounding boxes should not be confused with exact segmentation of an infected surface. They indicate approximate rectangular locations and may include surrounding healthy material. The suitability of object detection therefore depends on the level of localization required by the application.

2.2.4 YOLO-Based Object Detection

The You Only Look Once (YOLO) family of models was developed for object detection, combining the prediction of object locations and classes within an efficient inference process. Different versions of YOLO have introduced changes to model architecture, training methods, and deployment capabilities.

YOLO-based approaches are relevant to agricultural monitoring because they can be used to locate and classify visible objects or regions in photographs. Their suitability for a particular application must nevertheless be established through evaluation. Detection depends on factors such as dataset quality, the size and visibility of target regions, model configuration, and the hardware used for inference.

The present research uses YOLOv8 as the proposed detection framework. Its role is to analyze images of oyster mushroom cultivation bags and identify regions associated with the selected green mold and black mold categories.

The model's predictions are assessed using standard object-detection concepts. Precision indicates the proportion of positive detections that are correct, while recall indicates the proportion of relevant ground-truth instances that the model successfully detects. Intersection over Union (IoU) measures the overlap between a predicted bounding box and its corresponding ground-truth box. Mean Average Precision (mAP) can also be used to summarize detection performance across classes and defined IoU criteria.

These measures provide different information. High precision does not necessarily imply high recall, and an accurate class prediction does not automatically mean that the predicted bounding box is well positioned. Evaluating a detection model therefore requires attention to both classification and localization.

For the proposed application, response time is another relevant consideration. The model must return predictions within a reasonable period for a mobile-assisted inspection workflow. However, the actual response time depends on the deployment arrangement, image processing, network communication, and available computing resources. It must be measured rather than assumed from the choice of YOLOv8.

2.2.5 Rack-Level Contamination Tracking

Image analysis can identify a suspected infection, but farm management also requires information about where the affected cultivation bag is located. In a growing facility containing several racks and shelves, an isolated image prediction may be difficult to use unless it is associated with a physical location.

Rack-level contamination tracking addresses this requirement by connecting detection results with identifiers representing the relevant cultivation area. Depending on the implemented system, these records may include a rack identifier, cultivation-bag identifier, detection category, date, and user-confirmed health status.

The main purpose of maintaining these records is to support repeated inspection and provide a history of reported contamination. Farmers may then review which areas have been flagged and determine where additional checking is required.

It is important to distinguish contamination tracking from predicting the biological spread of disease. Recording the locations of suspected infections can provide useful observations over time, but it does not independently establish how an infection moved between bags or prove that one contaminated bag caused another infection.

2.3 Related research

Research on oyster mushroom cultivation has increasingly explored the use of digital technologies to improve farm monitoring and disease management. However, the studies differ in their main objectives, methods and level of automation.

Surige et al. [1] investigated an IoT-based monitoring system for oyster mushroom farms in Sri Lanka. Their work considered environmental monitoring alongside functions such as disease detection, harvest-time identification and yield prediction. This provides a useful foundation for applying technology to mushroom cultivation. However, the approach described in the proposal does not establish how individual contamination cases can be recorded and traced across specific cultivation racks.

Shilpashree et al. [2] explored automated disease detection in oyster mushroom cultivation using YOLOv5. Their work is relevant because object detection can identify and localize visible disease symptoms within an image. Nevertheless, the proposed study focuses on YOLOv8 and its integration into a mobile-assisted workflow for detecting green and black mold. This makes the implementation and its practical use in contamination recording important areas of investigation.

Raghu Kumar et al. [3] discussed smart mushroom farming through environmental control and image-based disease detection. This work highlights the potential of combining monitoring technologies with image analysis. However, the information available in the proposal describes the study as primarily conceptual, so it does not provide sufficient evidence to determine how well such an approach performs in a practical, rack-level disease-tracking process.

Other studies have examined low-cost monitoring systems for mushroom cultivation. Guragain et al. [4] explored an IoT-based ecosystem, while Al-Talib et al. [5] described a smart mushroom-growing kit using an ESP32 microcontroller and the

Blynk platform. These studies are relevant to the wider development of connected mushroom farms, particularly for monitoring growing conditions. Their primary emphasis, as described in the proposal, is on cultivation monitoring rather than the specific image-based identification and recording of green and black mold.

2.4 Existing systems products and techniques

Existing approaches to mushroom disease management can be considered in terms of how they identify contamination, monitor cultivation conditions and record affected areas.

Manual inspection relies on growers visually examining mushroom bags for changes in color, texture or growth. It is straightforward to carry out, but the results depend on the observer's experience and the conditions under which the inspection takes place. Recording findings consistently across multiple racks can also become difficult as the cultivation area grows.

Environmental monitoring systems use sensors and connect devices to observe conditions such as temperature and humidity. Systems based on IoT platforms can help growers monitor the cultivation environment and identify conditions that may affect mushroom growth. However, environmental readings alone cannot confirm whether a particular bag contains green or black mold. Image-based examination is therefore needed when the objective is to identify visible contamination.

Image classification techniques use machine-learning models to assign an image to a category. They can be useful when the objective is to classify an entire image as healthy or diseased. However, classification alone does not normally provide the location of the suspected mold within the image, which can limit its usefulness when users need to inspect areas.

Object detection techniques, including the YOLO family of models, identify objects and indicate their locations using bounding boxes. This is relevant to mushroom disease detection because it allows the system to highlight suspected contamination in an image rather than only returning a class label. The selected YOLOv8 model is used in this research to detect the targeted mold categories.

The proposed component combines object detection with a mobile-assisted workflow. Images are submitted to a backend developed using FastAPI, where the trained model performs inference and returns prediction results. The application can then use these results to support the identification and recording of suspected contamination. This combination is intended to connect image-level detection with rack-level tracking, rather than treating model prediction as the entire disease-management process.

2.5 Gap synthesis and conceptual framework

The reviewed literature indicates that digital technologies can support mushroom cultivation through environmental monitoring and image-based analysis. Nevertheless, the sources examined for this research do not sufficiently establish how visible green and black mold can be detected and linked to individual cultivation racks through a practical mobile-assisted workflow.

Three issues inform the direction of this study. First, the system must distinguish the targeted mold categories from healthy mushroom images. Second, detection should provide a visual indication of the suspected contamination so that users can interpret the prediction. Third, the result needs to be connected to the relevant cultivation record to support the identification of affected bags or racks.

To address these requirements, the individual component combines a YOLOv8 object-detection model with a FastAPI backend and a mobile application. The model processes the submitted image and returns its predictions, including the detected category, bounding-box location and confidence score. The application can present

these results to the user and support the recording of contamination information against the relevant rack or bag identifier.

The conceptual framework follows four main stages:

- Input: An image of an oyster mushroom cultivation bag is captured or selected, with the relevant rack or bag identifier supplied where required.

- Processing: The image is passed to the backend, which uses the trained YOLOv8 model to identify the targeted categories.

- Detection output: The system returns the predicted category, location of the detection and confidence score for presentation to the user.

- Record management: The detection result can be associated with the relevant cultivation record to support rack-level contamination tracking.

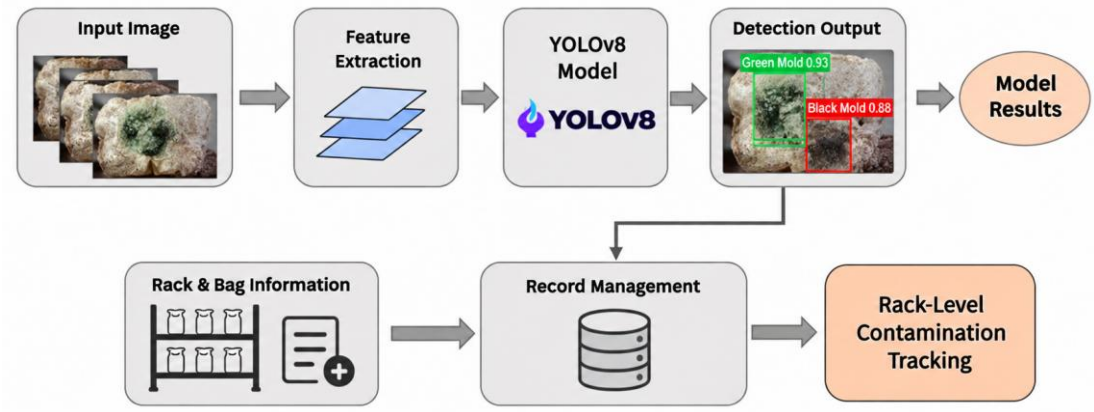


Figure 1 Conceptual Framework of the Mushroom Disease Detection and Rack-Level Tracking Component

The framework links automated image analysis with a practical record-management process. However, a prediction does not by itself confirm a biological diagnosis, and a failure to detect the two targeted mold categories does not necessarily prove that a mushroom bag is healthy. These limitations must be considered when interpreting the system's output and evaluating its suitability for farm use.

3 RESEARCH METHODOLOGY

3.1 Research design

This research followed an applied experimental approach to develop and evaluate an image-based mushroom disease detection component. The study focused on using computer vision to identify visible green and black mold in oyster mushroom cultivation bags and connecting the resulting predictions to a mobile-assisted contamination-tracking workflow.

The experimental part involved preparing image datasets, training a YOLOv8 object-detection model and examining its performance on images not used for training. The development part involved integrating the trained model with a FastAPI backend and the mobile application. This allowed the research to consider both the model's detection capability and its use within the intended application.

The study concentrated on three image categories: healthy mushrooms, green mold and black mold. The model's performance was assessed using object-detection evaluation metrics, while the integrated component was considered in terms of its ability to receive images, return predictions and support contamination-record management.

The research was limited to the selected image datasets and the two targeted mold categories. Therefore, the findings should be interpreted in relation to the available data and the implemented system rather than as evidence that the model can identify every possible mushroom disease.

3.2 Data participants or study materials

The primary study materials were digital images of oyster mushroom cultivation bags obtained from three sources. The datasets were brought together to support the development of a model capable of distinguishing healthy mushroom images from images containing the targeted mold categories.

Dataset category	Source	Reported image count
Healthy mushrooms	Kaggle	2,234
Black mold	Images collected at the Agricultural Research Center in Makandura	216
Green mold	Hugging Face	292

Table 1 Summary of Datasets Used

The healthy mushroom images were obtained from a Kaggle dataset originally labelled for harvest readiness. Since harvest-readiness labels do not automatically confirm the absence of disease, these images required particular attention during dataset preparation. Their suitability for the healthy class depends on whether the images were inspected and whether the original labels were appropriately removed or replaced. The original labels alone were not sufficient to establish that every image represented a healthy sample.

The black mold images were collected at the Agricultural Research Center in Makandura, while the green mold images were obtained from Hugging Face. The datasets differed in source and image characteristics, which made consistent labelling and preparation important before training.

The reported total of 2,742 represents the initial number of images, not necessarily the final number used for training and evaluation. The final dataset size depends on the images retained after inspection, preprocessing and removal of unsuitable or duplicate samples.

No human participants were required for the image-detection experiment as described. The main study materials were images, the trained machine-learning model, the application software and the outputs generated during evaluation.

3.3 Experimental or study procedure

The procedure was organized into dataset preparation, model training, evaluation and application integration.

Stage 1: Dataset inspection and preparation

The three image collections were examined and consolidated into a common dataset structure. The reported inspect_dataset.py script was used to examine dataset contents and formats, while prepare_dataset.py supported preprocessing and preparation for YOLOv8. The images and corresponding annotations needed to follow a consistent structure so that the training process could interpret the data correctly.

Attention was required when preparing for the healthy class because the original Kaggle labels concerned harvest readiness rather than disease status. The images needed to be checked and relabeled appropriately before they could be treated as healthy examples.

Stage 2: Dataset splitting

The intended split was 80% for training and 20% for validation and/or testing. Training images were used to update the model's parameters, while separate held-out images were needed to evaluate how well the model generalized to unseen examples.

The exact division of the 20% portion between validation and testing must be reported according to the actual experiment. If the full 20% was used only for validation, it should not be described as an independent test set.

Stage 3: Model training

The prepared dataset was used to train a YOLOv8 object-detection model through the train.py script. During training, the model learned visual patterns associated with the labelled categories and generated object-detection predictions. Training outputs, model weights and related performance artefacts were retained for inspection.

The trained weights were saved as best.pt, which was subsequently loaded by the backend for inference. The choice of YOLOv8 was based on its use in the implemented system. Any claim that it outperformed other architectures should be supported by comparable experiments using consistent datasets and evaluation settings.

Stage 4: Model evaluation

The trained model was evaluated using object-detection metrics and the available evaluation outputs. These were examined to understand the model's ability to identify the target categories and the types of errors it produced. The results were considered alongside the limitations of the image data and the possibility of missed or incorrect detections.

Stage 5: Application integration

The trained model was integrated into the backend through FastAPI. The inference.py module was responsible for loading the model and supporting prediction, while app.py exposed the backend endpoints.

The mobile application communicated with the backend through the API. The returned predictions could be presented to the user and used alongside rack or bag information for contamination tracking. This stage examined the practical connection between image analysis and the intended record-management workflow.

3.4 Measures and evaluation metrics

The study used object-detection metrics to evaluate the model. These metrics provide different perspectives on detection performance and should be interpreted together rather than relying on a single score.

Precision measures the proportion of predicted positive detections that are correct. Higher precision indicates fewer false-positive detections among the model's predictions.

$$\text{Precision}=TP/TP+FP$$

Recall measures the proportion of actual positive instances that the model successfully detects. Higher recall indicates that fewer relevant instances are missed.

$$\text{Recall}=TP/TP+FN$$

Here, (TP) represents true positives, (FP) represents false positives and (FN) represents false negatives, calculated according to the evaluation procedure used.

Intersection over Union (IoU) measures the overlap between a predicted bounding box and its corresponding ground-truth bounding box. It is used to assess whether a predicted location sufficiently matches the labelled object.

$$\text{IoU}=\text{Area of Overlap}/\text{Area of Union}$$

Mean Average Precision (mAP) summarizes average precision across the evaluated classes under a specified IoU evaluation setting. For this research, the reported mAP

value should identify the exact metric, such as mAP@50 or mAP@50:95, because these represent different evaluation criteria.

A confusion matrix was also considered to examine the distribution of correct and incorrect class predictions. Its values must be interpreted according to the axes, class labels and normalization used in the generated plot. A single value from a confusion matrix should not automatically be reported as overall model accuracy.

3.5 Data analysis

The analysis combined quantitative evaluation of the trained model with functional examination of the integrated component.

For the quantitative analysis, the model's evaluation outputs were examined using precision, recall, mAP and the confusion matrix. These measures were used to identify strengths and weaknesses in detecting the targeted categories. Where class-specific results were available, they were considered separately because the model may perform differently for healthy mushrooms, green mold and black mold.

The confusion matrix was used to examine which classes were correctly identified and where misclassification occurred. This was particularly important for interpreting predictions that could be mistaken for healthy samples. The analysis also needed to consider the number of images available for each category, since an uneven class distribution can influence how the overall results are interpreted.

For the functional analysis, the backend endpoints and prediction workflow were checked to determine whether images could be submitted and prediction responses returned. The intended connection between detection results and rack-level records was considered as part of the component's practical operation.

3.6 Ethics privacy security and responsible AI

Ethical considerations, data privacy and responsible use of artificial intelligence were considered throughout the development of the mushroom disease detection component. The research focused on images of oyster mushroom cultivation bags and aimed to support agricultural monitoring through automated image analysis.

Images collected at the Agricultural Research Center in Makandura were intended for research and system development. Their use was subject to the permissions and conditions applicable to data collection. Where images contained identifiable individuals or sensitive farm information, appropriate care was required to protect their privacy. Any permission obtained for collecting or using these images were considered part of the research process.

The datasets obtained from Kaggle and Hugging Face were also considered in relation to their original sources, licenses and usage conditions. Their sources were identified to support appropriate acknowledgement. Images used for demonstration purposes were distinguished from those used for model training and evaluation to avoid misrepresenting the experimental results.

Security considerations were considered when connecting the mobile application to the FastAPI backend. The backend address was configured through an environment variable so that it could be changed when required. The use of environment files also helped separate configuration settings from the main source code. Sensitive configuration values were not intended to be exposed in a public code repository. Additional security measures, such as access controls and secure communication, remained relevant to any deployment beyond the development environment.

The limitations of the trained model were also considered when interpreting its predictions. The component was designed to detect green and black mold, so its ability

to identify other diseases or unfamiliar visual patterns was limited. A high confidence score did not establish a biological diagnosis or indicate the severity of contamination. Similarly, the absence of a detected target category did not necessarily mean that a mushroom bag was healthy.

3.7 Reproducibility and research artefacts

The main research artefacts were retained to document the model development process and support future reproduction. These included the dataset inspection and preparation scripts (inspect_dataset.py and prepare_dataset.py), the training script (train.py), the trained model weights (best.pt), and the backend files (inference.py and app.py). Training logs, evaluation plots and confusion-matrix outputs were also retained where available.

Reproducing the experiment required access to the original datasets, their class labels, the preprocessing procedure, the dataset split, the training configuration and the software dependencies. The Python virtual environment (.venv) was used to manage the project's dependencies separately. Details such as the YOLOv8 model variant, training duration, image size and number of epochs needed to be obtained from the training configuration or logs to support a more accurate reproduction.

The source code and associated artefacts were maintained within the project files for development and submission. Access to the original datasets was subject to their respective availability and licensing conditions. These materials provided a record of the development process and a basis for further evaluation and improvement of the mushroom disease detection component.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

4.1.1 Functional Requirements

- FR1: The system shall accept an image of an oyster mushroom cultivation bag for analysis.

- FR2: The system shall process the submitted image using the trained YOLOv8 model.

- FR3: The system shall identify the targeted categories: healthy mushrooms, green mold and black mold, subject to the model's trained classes.

- FR4: The application shall support associating detection results with relevant rack or bag information where the corresponding record-management functionality is implemented.

4.1.2 Non-functional Requirements

- Performance: Image processing should return results within a practical time for the intended workflow.

- Reliability: The backend should handle image requests and return understandable responses when processing succeeds or fails.

- Maintainability: Dataset preparation, model inference, API handling and data-access logic should be organized into separate modules.

- Usability: Detection results should be presented in a way that users can interpret.

- Security: Configuration values and stored information should be handled carefully, particularly if the system is deployed outside the development environment.

- Extensibility: The design should allow future improvements to the model, support classes and deployment environment.

4.2 Component architecture within the complete solution

The individual component formed part of the wider mushroom farm monitoring project. Its main responsibility was image-based disease detection and support for rack-level contamination tracking, rather than environmental monitoring or automated control of cultivation conditions.

The component followed a client-server architecture. The mobile application acted as the client, while the FastAPI backend handled incoming requests and connected them to the trained machine-learning model. The database-related modules supported the application's data-management operations.

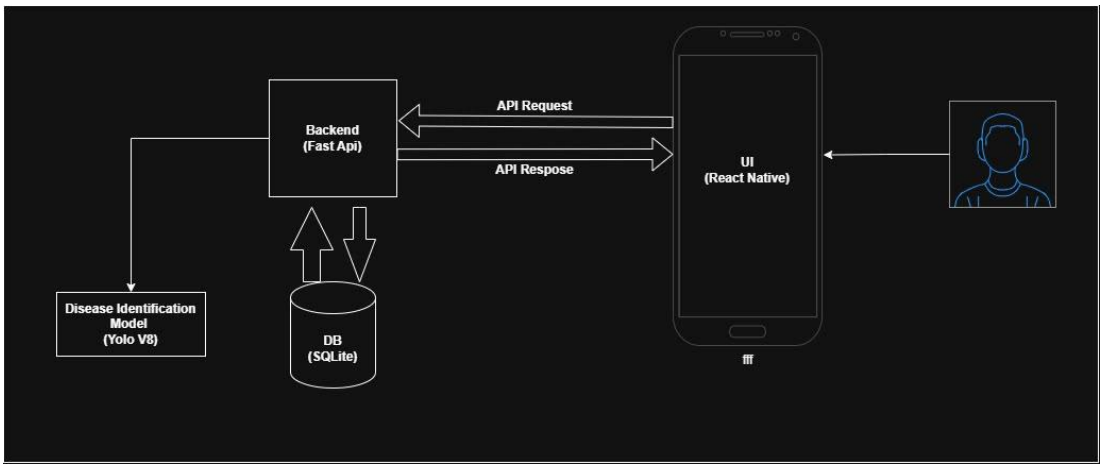


Figure 2 Component Architecture of the YOLOv8-Based Mushroom Disease Detection System

- User: Interacted with the system through the mobile application to access the disease detection functionality.

- Mobile application (React Native): Provided the user interface for interacting with the system and submitting requests to the backend.

- Backend (FastAPI): Handled API requests from the mobile application and coordinated communication with the disease identification model and database.

- Disease identification model (YOLOv8): Analyzed submitted mushroom images to identify the targeted mold categories and generate detection predictions.

- Database (SQLite): Stored and managed the data required by the component through the backend's database operations.

The mobile application communicated with the FastAPI backend through a REST API. The backend processed incoming requests and returned responses to the application. It also interacted with the YOLOv8 model to obtain image predictions and with the SQLite database to perform the required data operations.

4.3 Design alternatives and decisions

The design decisions were guided by the need to detect visible mushroom contamination and make the results available through the mobile application.

Model selection: Object detection was selected because it provides both a predicted category and the location of the detected object. YOLOv8 was used in the final implementation to identify the targeted mold categories. Other model approaches and YOLO versions were explored during development, but any claim of superior performance should be supported by comparable evaluation results.

Backend framework: FastAPI was used to expose the prediction functionality through API endpoints. This allowed the mobile application to communicate with the model through the backend rather than performing inference directly within the application.

Database selection: SQLite was used for data storage. Its use kept the database setup within the project environment without requiring a separate database server. The suitability of this choice depended on the scale and deployment requirements of the application.

Configuration management: The backend address was managed through an environment variable. This allowed the address to be changed between development environments without modifying multiple source files.

4.4 Detailed design

4.4.1 Model Inference

The trained YOLOv8 weights were saved in best.pt and loaded by the backend for prediction. When an image was submitted for analysis, the inference process passed it to the model and obtained the available detection results. These results included the predicted class, bounding-box coordinates and confidence score, depending on the prediction output.

The confidence threshold controlled which predictions were returned according to their confidence scores. It did not represent the percentage of contamination in a mushroom bag. Since the model focused on healthy mushrooms, green mold and black mold, it could not reliably identify diseases outside its trained categories.

4.4.2 API Communication

The backend exposed endpoints for communication and basic service checks. The mobile application communicated with these endpoints using API requests. This approach kept the model-loading and prediction logic on the backend and allowed the frontend to receive results without directly handling the trained model weights.

4.4.3 Database Operations

SQLite was used to manage the component's database operations. The files db.py, domain.py and repository.py supported the database-related structure, according to their respective responsibilities in the implementation.

The database interaction was handled through the backend rather than directly by the mobile application. Where implemented, the stored information could be associated with the relevant rack or bag to support contamination tracking. The actual database fields and relationships should be documented according to the implemented schema.

Entity	Main fields
Rack	Rack ID; total bags; current contamination status; last updated time
Bag	Bag ID; rack ID; current status (healthy, green mold, black mold)
Detection record	Image reference; rack ID; bag ID; capture timestamp; predicted class; bounding box and confidence; optional notes
Contamination log	Rack ID; bag ID; contamination type; date and time; detection reference

Table 2 Information stored by the component

4.4.4 Configuration and Integration

The backend address was configured through an environment variable, allowing it to be updated when the development environment changed. The Python virtual environment (.venv) was used to keep project dependencies separate from the system's general Python installation.

4.4.5 Interaction Sequence

The farmer captures an image and enters the rack and bag IDs (step 1). The app submits the image and metadata to the backend (2), which requests a contamination detection from the YOLOv8 engine (3). Related rack records are retrieved or stored (4–5), and the class prediction and detection result are returned to the backend (6). The backend stores the result with timestamp, rack ID, bag ID and class (7–8), updates the rack-level contamination status (9–10), and returns the detection result and contamination status to the app (11), which displays it to the farmer (12).

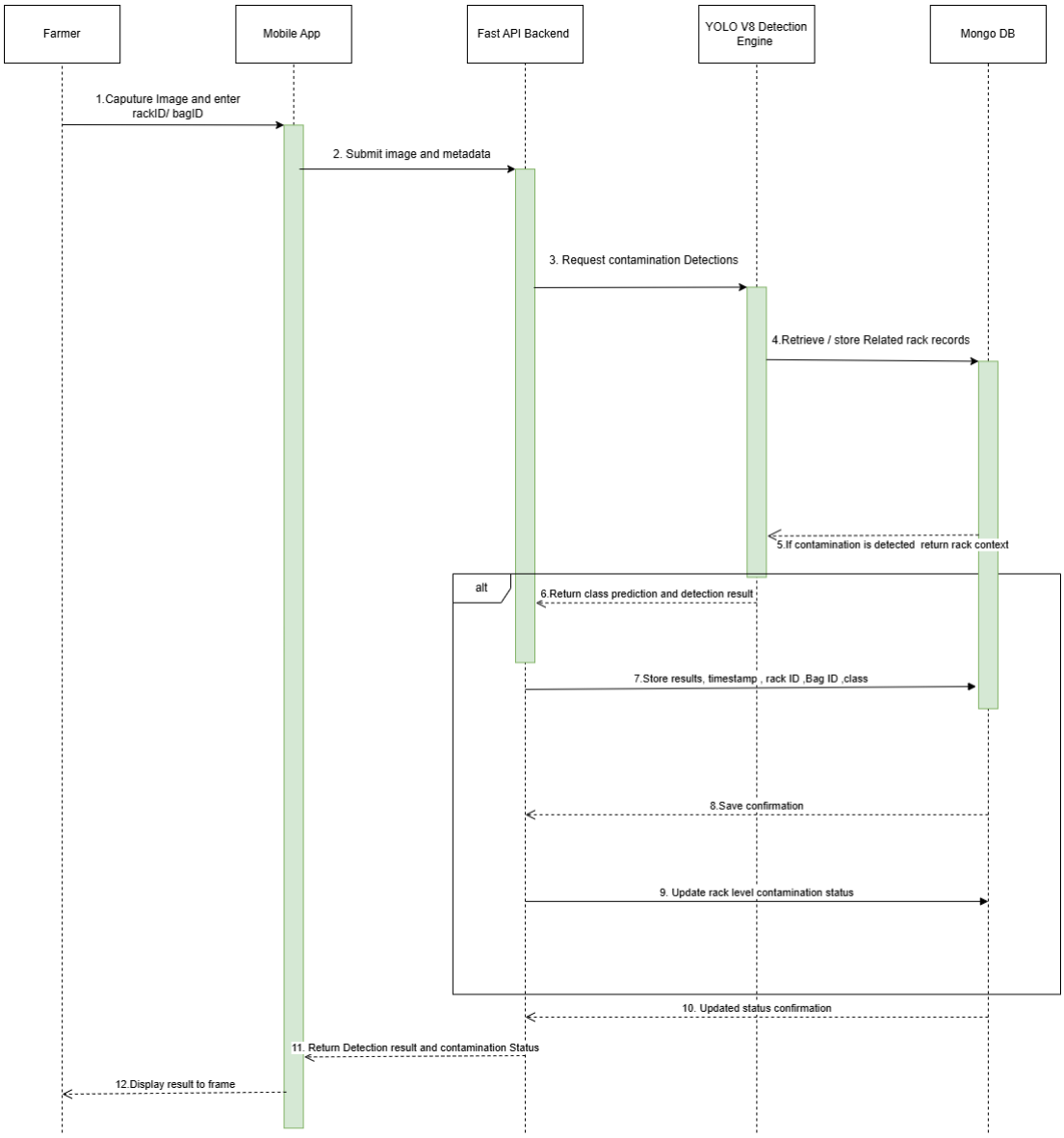


Figure 3 Sequence diagram of image submission, detection and rack-level update

4.5 Standards and professional practice

The component was developed with attention to modularity, maintainability and responsible handling of research data. The API separated communication with the mobile application from the model's prediction logic, while the backend files organized inference and database-related responsibilities.

The use of a Python virtual environment helped isolate project dependencies, and environment-based configuration reduced the need to hard-code the backend address throughout the source code. These practices supported a more manageable development process.

External datasets were considered in relation to their original sources, labelling and applicable usage conditions. Appropriate acknowledgement was necessary, and the suitability of the labels was important for avoiding misleading training results. The model's performance also needed to be reported using clearly identified evaluation metrics supported by the experimental outputs.

Responsible AI practices were relevant because incorrect predictions could affect how users interpreted contamination. The model's outputs were therefore understood as predictions rather than confirmed biological diagnoses. Its limitations, including missed detections and its restricted range of disease categories, need to be acknowledged.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

Technology	Purpose
Python	Model development and backend processing
YOLOv8	Oyster Mushroom disease detection
FastAPI	Backend API development
React Native	Mobile application interface
SQLite	Data storage

Table 3 Technologies used in the component

The Python virtual environment helped keep project dependencies separate from the system-wide Python installation. The backend address was configured through an environment variable so that it could be changed when the development environment changed.

5.2 Component implementation

5.2.1 Model Preparation and Training

The image datasets were prepared for use with YOLOv8 before training. The inspect_dataset.py script was used to examine dataset contents and formats, while prepare_dataset.py supported the preparation process. The train.py script was used to train the model.

The trained weights were saved as best.pt. This file was subsequently used by the backend to generate predictions for submitted images. Training outputs and evaluation plots were retained where available to support the examination of model performance.

5.2.2 Backend Implementation

The backend was implemented using FastAPI. The app.py file defined the API endpoints, while inference.py handled model loading and prediction-related operations.

The backend exposed three main endpoints:

- GET / provided a basic endpoint for checking whether the backend was responding and whether the model was loaded, according to the implemented response.

- GET /health provided backend and model health information.

- POST /predict accepted an image for analysis and returned the available prediction results.

The prediction response included information such as the detected category, bounding-box coordinates and confidence score, depending on the output returned by the model. The confidence threshold was used to filter predictions and did not represent the amount or severity of contamination.

5.2.3 Database Implementation

SQLite was used for database operations. The backend included db.py, domain.py and repository.py to organize database-related functionality and data access. This arrangement separated database operations from the model inference process. The actual data stored and the relationships between records depended on the implemented database schema. Where supported by the application, rack and bag information could be used to associate detections with the relevant cultivation location.

5.2.4 Mobile Application Interaction

The React Native application provided the interface through which users interacted with the detection functionality. The application communicated with the backend using API requests instead of loading the YOLOv8 model directly within the mobile interface.

The backend address was configurable through an environment variable. This was useful during development because the address could change depending on the device and network configuration. The application depended on the backend being reachable to obtain prediction results.

On the capture screen the user opens the camera or selects a gallery image, enters the rack ID, bag ID and optional notes, and presses Submit for Detection. The tracking screen shows a selected rack with total, healthy and infected bag counts and a grid of bags colored by status. The history and reports screen lists past detections with thumbnails, supports search by rack or bag ID and filtering by class, shows summary totals, and exports a report.

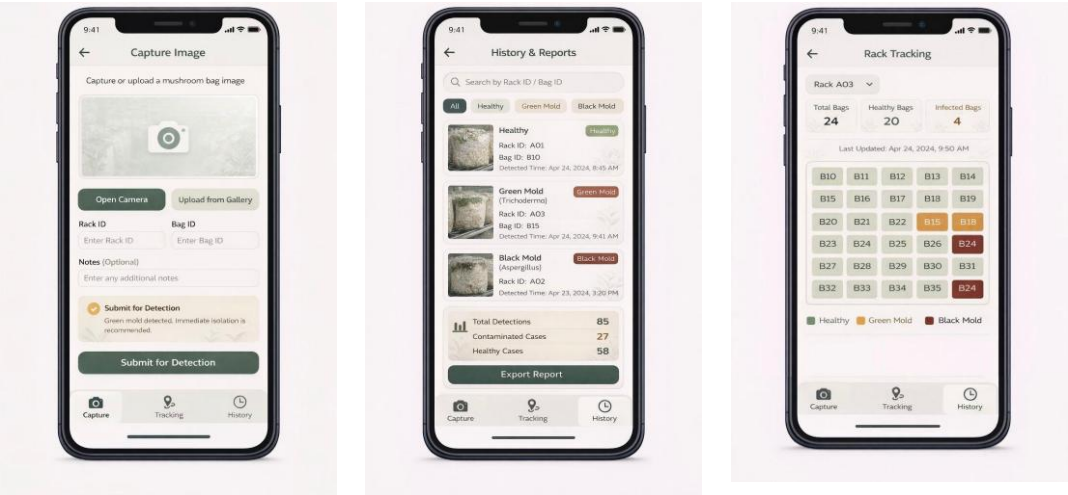


Figure 4 Preliminary UI mockups: (a) Capture Image, (b) History and Reports, (c) Rack Tracking

5.3 Integration with the other components

The disease detection component formed part of the wider Rack-Level Micro-Climate and Growth Optimizer for Oyster Mushroom Farms.

The growth stage component provided information about mushroom development, while the moisture stress component assessed environmental conditions using temperature and humidity data. The CO_2 stress component focused on identifying potential physiological stress through carbon dioxide readings and image-based validation. The disease detection component addressed a separate requirement by identifying visible green and black mold in mushroom images. Together, these components addressed different aspects of mushroom cultivation.

5.4 Verification and software testing

Verification focused on checking the operation of the disease detection component and its communication with the backend. The FastAPI documentation interface provided a way to send requests to the available endpoints and inspect their responses.

The main checks covered backend availability, model loading, image submission and prediction responses. The / and /health endpoints supported basic service checks, while /predict was used to submit images for analysis. Database operations also required verification of where they formed part of the implemented workflow.

Model performance was assessed separately using precision, recall, mean Average Precision (mAP) and the confusion matrix. These measures helped evaluate detection performance, whereas functional tests examined whether the software operated as intended.

The final testing results should be recorded according to the checks performed. Any failures, unexpected predictions or integration issues should be documented alongside the corrective actions taken. Since the model was trained to identify specific

categories, testing also needed to consider the possibility of missed detections and incorrect classifications.

5.5 Deployment and operation

During development, the component operated through the React Native application and the FastAPI backend. The backend loaded the trained best.pt model for image prediction and returned the available results to the application. SQLite supported the database operations implemented within the backend.

The backend address was managed through an environment variable, allowing it to be changed when the development environment changed. Operation depended on the backend being accessible, the required Python packages being installed, the model weights being available and the database being configured correctly.

Deployment through Azure or AWS was considered as a future possibility. Further deployment work would require appropriate configuration, access controls and protection of stored information.

6 RESULTS AND VALIDATION

6.1 Research results

The research examined the use of YOLOv8 for identifying healthy oyster mushrooms and detecting visible green and black mold. The model was trained using the prepared image dataset, and its performance was assessed using object-detection evaluation metrics.

Evaluation metric	Reported result
Precision	96.2%
Recall	94.8%
mAP@50	95.5%

Table 4 Reported Performance of the Disease Detection Model

Precision measured the proportion of predicted detections that were correct, while recall measured the proportion of actual target instances that were detected. The mAP@50 value summarized detection performance using an Intersection over Union threshold of 0.50.

The normalized confusion matrix reported that 93% of background instances were correctly assigned to the background category, while 7% were incorrectly classified as black mold. This suggested that some background patterns could be mistaken for disease, particularly where dark areas or shadows resembled mold.

6.2 Product and technical results

Technical measure	Provisional target
Model loading time	Within 10 seconds
Average image prediction response time	Within 3 seconds per image
Successful valid-image requests	At least 95%
Invalid-image handling	Clear error response or rejection

Health-check endpoint	Valid status response

Table 5 Product and Technical Performance Results

6.3 User or stakeholder validation

User acceptance testing involved 15 growers, comprising five experienced mushroom farmers and ten novice growers. The complete platform was evaluated using a five-point Likert scale to assess its usability and practical value.

The disease-quarantine response category received a mean score of 4.80 out of 5, corresponding to a satisfaction rate of 96.0%. The overall operational-value category received a mean score of 4.87 out of 5, with a satisfaction rate of 97.4%.

These findings indicated positive stakeholder perceptions of the system, particularly its potential to support the identification and isolation of affected cultivation bags. However, the evaluation covered the complete platform, and the results did not represent a separate assessment of the disease detection component alone.

6.4 Integration results

Integration testing confirmed that the components communicated successfully within the overall system. The disease-detection component contributed information about visible contamination, while the other components provided information related to mushroom growth and environmental stress. Combining these functions enabled the system to provide a more comprehensive view of mushroom cultivation conditions at rack level.

6.5 Summary of findings

This chapter presents the results and evaluation of the oyster mushroom disease-detection component and its role within the integrated system. The YOLOv8 model was used to identify green and black mold in mushroom images, with the reported

project results indicating a precision of 96.2%, recall of 94.8%, and mAP@50 of 95.5%.

The component was implemented using a FastAPI backend and SQLite database, enabling image processing and supporting database operations. User acceptance testing involving 15 mushroom growers also indicated positive perceptions of the overall system, with a mean score of 4.80 out of 5 for disease-quarantine response and 4.87 out of 5 for overall operational value.

Integration testing confirmed that the four components worked together to support disease identification, growth-stage monitoring, moisture-stress assessment, and CO_2 stress prediction. However, the disease-detection component remains limited to its trained classes, meaning that failure to detect green or black mold does not necessarily confirm that a mushroom bag is healthy.

7 DISCUSSION

7.1 Interpretation of findings

The findings indicate that image-based object detection can support the identification of visible fungal contamination in oyster mushroom cultivation bags. The use of YOLOv8 allowed the system to identify green and black mold and indicate their locations within an image. This is useful because farmers need more than a general indication that a bag may be diseased; knowing where contamination appears can help them inspect affected areas and decide whether further action is necessary.

The results also suggest that the disease-detection component can be used as part of a wider cultivation monitoring system. By combining visual disease information with growth-stage and environmental monitoring outputs, the system can provide farmers with a broader understanding of cultivation conditions. However, the model's performance should be interpreted within the limits of its training data and target classes. Successful detection of the two selected mold types does not establish that the system can identify every disease affecting oyster mushrooms.

7.2 Comparison with literature baselines and existing solutions

Previous research has explored deep learning and IoT technologies for improving mushroom cultivation. Surige et al. [1] investigated an IoT-based monitoring system for Sri Lankan oyster mushroom farms using CNN and LSTM techniques. Shilpashree et al. [2] explored deep learning for mushroom disease detection using YOLOv5. These studies demonstrate the potential of combining image analysis and digital monitoring to support cultivation management.

The present study builds on this direction by applying YOLOv8 to the detection of two specific contamination categories: green mold associated with Trichoderma and black mold associated with Aspergillus niger. The emphasis on identifying contamination types and locating affected regions is intended to provide more actionable information than a broad healthy-or-diseased classification alone.

However, the reported performance values should not be treated as proof that YOLOv8 is universally superior to the models used in earlier studies. A fair comparison would require testing the models using comparable datasets, class definitions, evaluation procedures, and experimental conditions.

7.3 Research contribution

The research contributes to the application of computer vision in oyster mushroom cultivation by developing an image-based approach for detecting two selected forms of fungal contamination. It brings together image preparation, object detection, and evaluation within a workflow designed around the practical needs of mushroom growers.

Another contribution is the use of farm-related data alongside images obtained from external datasets. This provides a starting point for investigating disease detection in a Sri Lankan cultivation context.

7.4 Product contribution

The main product contribution is a mobile-assisted disease-detection component that connects a React Native interface with a FastAPI backend and a trained YOLOv8 model. Users can submit mushroom images for analysis and receive detection results through the application workflow. SQLite supports the component's database operations.

The component forms part of the wider Rack-Level Micro-Climate and Growth Optimizer for Oyster Mushroom Farms. Its role is to provide visual information about potential contamination alongside the other components' outputs concerning growth stages, moisture stress, and CO_2-related stress.

7.5 Limitations and threats to validity

Several limitations affect the interpretation of the findings. First, the model focuses on green and black mold. Other diseases, contamination types, or unusual visual conditions may not be recognized correctly. In particular, the absence of a detected target class should not automatically be interpreted as confirmation that a mushroom bag is healthy.

Second, the dataset combines images from different sources, including healthy mushroom images from Kaggle, black mold images collected at the Agricultural Research Center in Makandura, and green mold images obtained from Hugging Face. Differences in image quality, lighting, background, and collection conditions may affect the model's ability to generalize to new farm environments. The labels assigned to the healthy images also require careful verification because the original dataset was not necessarily designed to identify disease-free mushroom bags.

Third, the reliability of the reported metrics depends on the dataset split and evaluation procedure. If highly similar images appear in both training and evaluation subsets, the results may overestimate performance on genuinely new images. The reported results therefore need to be interpreted considering the dataset preparation process and the independence of the evaluation data.

7.6 Implications

The findings have practical implications for mushroom growers and future agricultural technology development. A mobile-assisted detection tool could help growers identify suspicious contamination earlier, inspect affected cultivation bags, and make more informed decisions about isolation or further examination. When combined with growth and environmental monitoring, disease-related information may also help growers interpret cultivation problems more comprehensively.

For developers and researchers, the study demonstrates the importance of treating dataset quality, disease-specific labels, model evaluation, and application integration as connected parts of an agricultural AI solution. A model with strong evaluation results is not sufficient on its own; the predictions must also be understandable and useful within the user's workflow.

Further development should prioritize broader farm-based validation, verification of healthy and contaminated image labels, and testing across different lighting conditions, camera devices, and cultivation stages.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

This research focused on developing an image-based disease-detection component for oyster mushroom cultivation. The main aim was to identify visible signs of green and black mold using deep learning and make the results accessible through a mobile-assisted application. The developed solution combined a YOLOv8 model with a FastAPI backend, allowing users to submit mushroom images and receive detection results.

The work demonstrates how computer vision can be applied to a practical agricultural problem. Rather than relying entirely on visual inspection, growers can use the system as an additional source of information when examining cultivation bags for possible contamination. Its inclusion in the wider mushroom monitoring platform also provides an opportunity to consider disease-related information alongside growth and environmental conditions.

Although the reported model results indicate promising detection performance, further validation is needed before the system can be considered dependable across different farms and cultivation conditions. The study therefore provides a useful foundation for image-based mushroom disease monitoring, while recognizing that automated predictions should support, rather than replace, careful inspection and appropriate disease-management decisions.

8.2 Achievement of objectives

The dataset preparation objective was addressed by combining healthy mushroom images from Kaggle, black mold images collected at the Agricultural Research Center in Makandura, and green mold images obtained from Hugging Face. These datasets provided image samples used to develop the detection model. However, the reliability of the healthy class depends on verifying that the assigned labels accurately represent disease-free samples.

The disease detection objective was achieved through the development of a YOLOv8 model targeting green and black mold. The reported results included 96.2% precision, 94.8% recall, and 95.5% mAP@50. These results indicate promising detection performance for the evaluated dataset, although further testing with independent farm images is needed to establish how well the model generalizes to unfamiliar conditions.

The mobile image-processing objective was addressed by connecting the React Native application to the FastAPI backend. The backend processes submitted images using the trained model and returned detection results to the application. This provides a practical workflow through which users can obtain predictions from mushroom images.

The rack-level contamination-tracking objective was supported through the component's integration into the wider mushroom monitoring system. Integration testing confirmed that the four components worked together to support disease detection, growth-stage monitoring, moisture-stress assessment, and CO_2 stress prediction. The extent of contamination tracking depends on the implemented recording and retrieval of rack and bag identifiers.

Finally, the system evaluation objective was addressed through model-performance evaluation and functional and integration testing. User acceptance testing involving 15 growers also produced positive overall platform ratings, including a mean score of 4.80 out of 5 for disease-quarantine response. However, these user ratings reflect the complete platform rather than the independent performance of the disease-detection component.

8.3 Individual research and product contributions

The individual research contribution was the investigation and application of YOLOv8-based object detection for identifying two selected forms of contamination

in oyster mushroom cultivation: green mold associated with Trichoderma and black mold associated with Aspergillus niger. The work involved preparing image data, developing the detection model, and examining its performance using object-detection evaluation measures.

The individual product contribution was the disease-detection workflow connecting the React Native mobile application to a FastAPI backend and the trained model. SQLite was used for database operations within the component. This provided a practical route for processing mushroom images and presenting detection outputs through the application.

8.4 Future directions

Future work should first improve the quality and coverage of the image dataset. Additional samples should be collected from different farms, cultivation stages, lighting conditions, and camera devices. Healthy samples should be verified carefully, and disease labels should be checked to ensure that the model is trained and evaluated using reliable ground truth.

The model should then be evaluated using a clearly separated, independently labelled test dataset. Further experiments could examine performance for each disease class, false detections, missed infections, and prediction time under realistic usage conditions. Comparing model configurations under the same experimental setup would also provide stronger evidence for selecting an appropriate model.

Another direction is to strengthen the rack-level tracking functionality. Future versions could improve the recording and retrieval of rack identifiers, bag identifiers, detection history, and changes in contamination status, where these features are not already fully implemented. This would help growers follow up on previously identified cases instead of treating each image as an isolated prediction.

Deployment planning should also consider connectivity, data protection, maintenance, and the resources required to operate the system consistently. These steps would help determine its practical value beyond the initial development and evaluation stage.

REFERENCES

[1]

Y. D Surige, P. W. S. M, G. P. K. N, A. K. P. W, N. Gamage, and D. Nawinna, "IoT-based Monitoring System for Oyster Mushroom Farming," *IEEE Xplore*, Dec. 01, 2021. https://ieeexplore.ieee.org/abstract/document/9671112.

[2]

W. S. Nuankaew, P. Sombutthai, W. Monkhuan, T. Sararat, and P. Nuankaew, "Harnessing AI for Agriculture: Oyster Mushroom Disease Detection With IoT and Web Application on Growing Bags Using Deep Learning," *2025 Joint International Conference on Digital Arts, Media and Technology with ECTI Northern Section Conference on Electrical, Electronics, Computer and Telecommunications Engineering (ECTI DAMT & NCON)*, pp. 606–611, Jan. 2025, doi: 10.1109/ectidamtncon64748.2025.10962000.

[3]

Raghu kumar B S, Yogesh H C, Priya MM, Muktha S P Raj, and Mohammed Ikram, "IOT-Driven Smart Farming: Enhancing Mushroom Cultivation With Environmental Control and Image-Based Disease Detection," *International Journal of Science and Research Archive*, vol. 16, no. 3, pp. 396–401, Sept. 2025, doi: 10.30574/ijsra.2025.16.3.2565.

[4]

D. P. Guragain, B. Shrestha, and I. Bajracharya, "A Low-Cost Centralized IoT Ecosystem for Enhancing Oyster Mushroom Cultivation," *Journal of Agriculture and Food Research*, vol. 15, p. 100952, Mar. 2024, doi: 10.1016/j.jafr.2023.100952.

[5]

A. A. Al-Talib, C. Kuan, N. I. M.Tahir, A. A. Mustapha, and Y. H. T., “IoT Based Smart Mushroom Growing Kit,” *Proceedings of International Conference on Artificial Life and Robotics*, vol. 29, pp. 789–794, Feb. 2024, doi: 10.5954/icarob.2024.os25-7.

GLOSSARY

Bounding Box - A rectangular area drawn around a detected object or suspected contamination region in an image.

Contamination Tracking - The process of recording and monitoring suspected contamination in mushroom cultivation bags and linking the records to relevant cultivation locations.

Confidence Score - A model-generated score indicating confidence associated with a detection prediction. It does not indicate the severity of contamination.

Dataset Annotation - The process of assigning labels and, where required, location coordinates to images so they can be used to train and evaluate an object-detection model.

Dataset Preprocessing - The preparation of raw image data and annotations into a consistent format suitable for model training and evaluation.

False Negative - A relevant object or contamination instance that the detection model fails to identify.

False Positive - An incorrect detection in which the model identifies an object or contamination category that is not present according to the reference annotation.

Fungal Contamination - The presence or suspected presence of unwanted fungi in mushroom cultivation materials that may affect cultivation health and productivity.

Green Mold - A contamination category associated with Trichoderma and targeted by the disease-detection component developed in this research.

Ground-Truth Annotation - The reference label or bounding box assigned to an image during dataset preparation, used to assess the correctness of model predictions.

Healthy Class - The dataset category intended to represent mushroom cultivation bags without the targeted visible contamination.

Inference - The process of using a trained model to generate predictions for a new image.

Localization - The identification of the approximate position of a detected object or suspected contamination region within an image.

Model Training - The process through which a machine-learning model learns patterns from labelled training data.

Precision - The proportion of predicted positive detections that are correct. It indicates how many reported detections are correct.

Recall - The proportion of relevant ground-truth instances successfully detected by the model. It indicates how many actual target instances were identified.

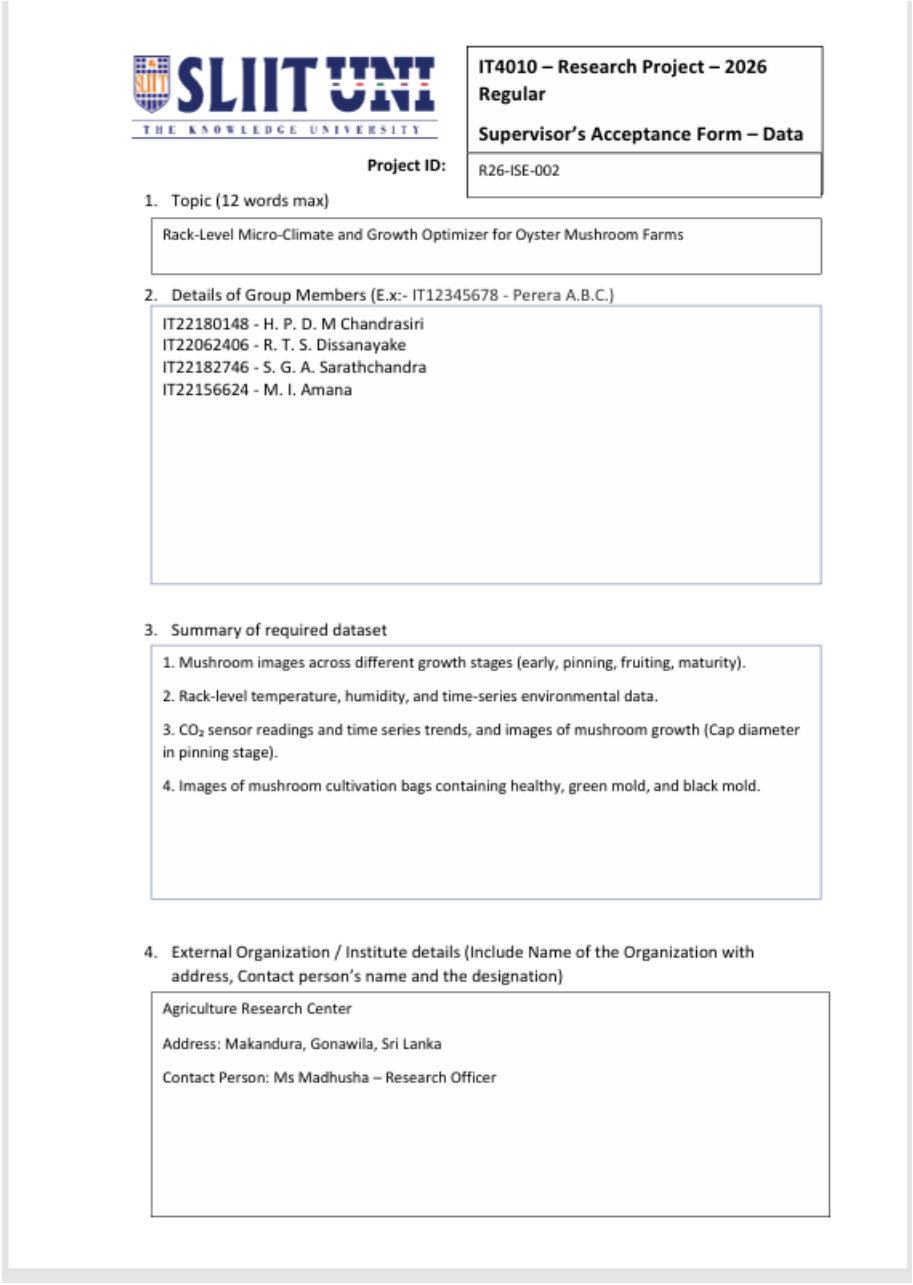
Test Dataset - A separate set of labelled images used to evaluate a trained model on examples not used to train it.

True Positive - A relevant object or contamination instance correctly detected by the model.

Validation Dataset - A set of images used during model development to assess performance and support model selection or tuning.

APPENDICES

Appendix A Supervisor's Acceptance Form - Data



SLIIT UNI
THE KNOWLEDGE UNIVERSITY

IT4010 – Research Project – 2026 Regular

Supervisor's Acceptance Form – Data

Project ID: R26-ISE-002

1. Topic (12 words max)

Rack-Level Micro-Climate and Growth Optimizer for Oyster Mushroom Farms

2. Details of Group Members (E.x:- IT12345678 - Perera A.B.C.)

IT22180148 - H. P. D. M Chandrasiri
IT22062406 - R. T. S. Dissanayake
IT22182746 - S. G. A. Sarathchandra
IT22156624 - M. I. Amana

3. Summary of required dataset

1. Mushroom images across different growth stages (early, pinning, fruiting, maturity).
2. Rack-level temperature, humidity, and time-series environmental data.
3. CO_2 sensor readings and time series trends, and images of mushroom growth (Cap diameter in pinning stage).
4. Images of mushroom cultivation bags containing healthy, green mold, and black mold.

4. External Organization / Institute details (Include Name of the Organization with address, Contact person's name and the designation)

Agriculture Research Center

Address: Makandura, Gonawila, Sri Lanka

Contact Person: Ms Madhusha – Research Officer

Figure 5 Supervisor's Acceptance Form – Data, Research Project 2026

Appendix B Preliminary Model Training and Evaluation Results

Before finalizing the disease-detection model, preliminary training experiments were conducted to examine the model's behavior on the initially prepared dataset. These experiments formed part of the model development process and helped assess the suitability of the dataset structure, class labels, and training configuration.

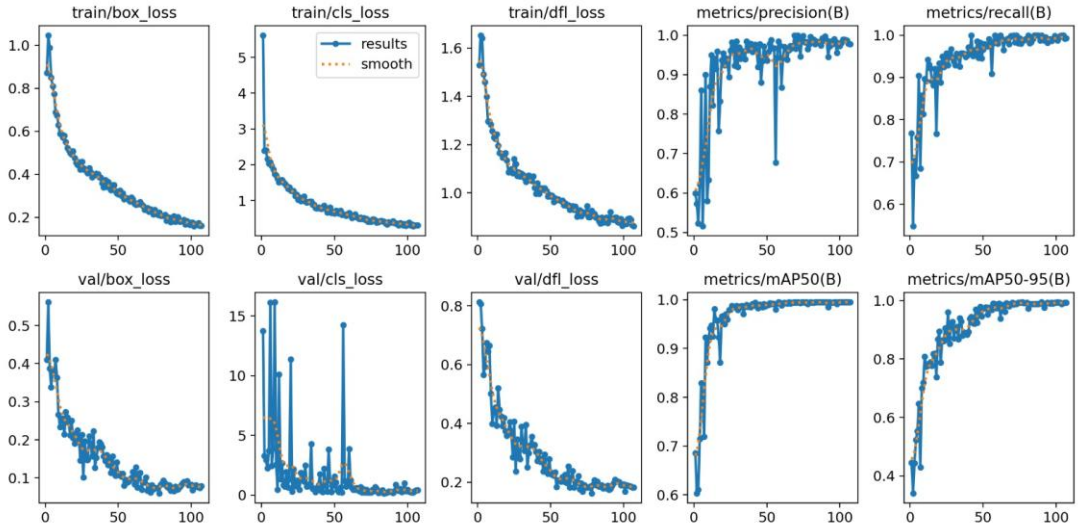


Figure 6 Preliminary Confusion Matrix

Appendix C Functional Test Cases

Test ID	Result	Status
TC01	The API received the uploaded mushroom image and returned a prediction response containing the detected class and associated confidence score, where a detection was produced.	Pass
TC02	The system rejected the unsupported file type or returned an error response indicating that the file could not be processed.	Pass

TC03	The mobile application submitted the image to the backend and displayed the prediction response received from the API.	Pass
TC04	The database stored the contamination record and allowed the saved record to be retrieved with the correct information.	Pass

Table 6 Summary of Functional Test Cases

Appendix D Academic integrity and AI use disclosure

ChatGPT, Gemini, Claude, and GitHub Copilot were used as supplementary tools for technical concept clarification, code generation and modification, debugging, and writing and documentation. AI-generated suggestions and code were subject to review and verification against the actual implementation and available evidence. The author retains full responsibility for the originality, accuracy, implementation, results, and conclusions presented in this thesis, in accordance with the IT4010 AI Usage Policy.